Study Notes: Days 1–7 — C# Fundamentals

These notes follow Week 1 of the senior .NET interview roadmap. The examples use modern C#; version-specific behavior is identified where it matters. They are illustrative, not claims about your own projects.

Day 1 — Value vs reference types, stack vs heap, boxing/unboxing, class vs struct

Month 1 — Foundations, Week 1 — C# Fundamentals.

Goal: Explain how values are stored, copied, passed, and compared, and choose a type based on its semantics —not simply on a presumed allocation location.

Value and reference semantics

A **value-type variable** contains its value. Assignment or a normal method call copies that value. A value type can still contain references: copying the struct copies those references, not the objects they reference.

A **reference-type variable** contains a reference to an object, or `null`. Assignment copies the reference; both variables can then refer to the same object. C# passes parameters **by value by default**, even when the value being passed is a reference. Mutating the shared object is visible to the caller, but reassigning the callee's parameter is not. `ref`, `out`, and `in` provide different ways to pass a reference to a variable instead.

Stack and managed heap

The stack holds call-related storage, while the managed heap is where objects are generally allocated and tracked by the garbage collector. Avoid saying “value types are always on the stack.” An `int` field in a class and the elements of a struct array are stored inline in heap objects. A captured local or an async state-machine field may also reside in a heap object. The JIT can keep values in registers or optimize away allocations, so storage is an implementation detail rather than a rule for choosing a type.

Managed allocation is often fast, but allocation rate, object lifetime, and GC pauses matter in hot paths. Measure with production-like workloads before optimizing.

Boxing and unboxing

Boxing converts a value type to `object` or an implemented interface by creating a boxed representation and copying the value. For example, putting an `int` in an `ArrayList` boxes it. **Unboxing** checks the boxed type and extracts the value; it does not itself require another heap allocation. Nullable value types have special boxing rules: a null `int?` becomes a null reference, while a non-null one boxes the underlying `int`.

Generic collections such as `List<int>` store `int` values directly, avoiding boxing on ordinary additions. Boxing can still occur if a value later crosses an `object` or interface boundary. Not every interpolation boxes: behavior depends on the selected API and runtime/compiler implementation.

Class or struct?

Use a **class** when identity, shared mutable state, inheritance, or a potentially large object is central to the model. Use a **struct** for a small, logically value-like type that benefits from copying and preferably is

immutable. Large structs can make copies expensive; there is no universal byte-size cutoff, so measure where it matters.

`readonly struct` prevents assignment to its instance fields after construction and communicates value immutability. It is not necessarily *deeply* immutable: a readonly reference field can still point to a mutable object. The compiler can also avoid some defensive copies for readonly operations.

By default, classes inherit reference equality unless a type overrides it. Structs inherit value equality from `ValueType`; its implementation details and cost vary, so hot-path types should provide appropriate equality and hashing. `record` and `record struct` synthesize value-oriented equality, subject to their own rules.

A `ref struct`, such as `Span<T>`, has restrictions that prevent it from escaping into ordinary heap-stored contexts. It cannot be boxed or captured in a lambda. Starting with **C# 13**, ref-struct locals can be used in async methods where their lifetime does not cross an `await`; older C# versions prohibit such locals in async methods. Check language version and lifetime rules rather than giving an unconditional “never in async” answer.

Example

```
using System;
using System.Collections;

var first = new Point(1, 2);
var second = first;
second.X = 99;
Console.WriteLine(first.X); // 1: a separate value

var customer = new Customer { Name = "Ada" };
Rename(customer);
Console.WriteLine(customer.Name); // Grace: the shared object changed
Replace(customer);
Console.WriteLine(customer.Name); // Grace: only the local reference changed

object boxed = 42;            // boxes an int
int number = (int)boxed;       // checks and unboxes
var oldCollection = new ArrayList { number }; // boxes on insertion

static void Rename(Customer c) => c.Name = "Grace";
static void Replace(Customer c) => c = new Customer { Name = "New" };

struct Point
{
    public Point(int x, int y) { X = x; Y = y; }
    public int X;
    public int Y;
}

class Customer { public string Name { get; set; } = ""; }
```

Likely interview questions

1. What happens when you copy a struct that contains a reference field?
2. Why can a method mutate a class instance without using `ref`, but not replace the caller's reference?
3. Give two examples of value-type data stored in a heap object.
4. What allocations occur during boxing and unboxing?
5. When would you choose a class over a struct, and how would you validate the performance decision?
6. What does `readonly struct` guarantee, and what does it not guarantee?
7. How do default class and struct equality differ?
8. What restrictions apply to `ref struct`, and how does the C# version affect async usage?

Pitfalls and recap

- **Pitfalls:** Equating type categories with storage locations; calling unboxing an allocation; assuming readonly guarantees deep immutability; relying on an outdated blanket rule about `ref struct` in async code.
- **Recap:** Values copy their contents; references copy an alias. Parameter passing is by value unless declared otherwise. Choose types for semantics, then measure allocation and copy costs.

Day 2 — OOP: encapsulation, abstraction, inheritance, polymorphism

Month 1 — Foundations, Week 1 — C# Fundamentals.

Goal: Connect the four pillars to production design, invariants, and maintainability.

Encapsulation

Encapsulation groups state with the operations that protect it and hides implementation details. Its practical payoff is **preserving invariants**. For example, an account's balance should not become negative merely because another class assigned a public field; expose an operation that validates the withdrawal. A public property with an unrestricted setter is syntactically encapsulated but may not protect the business rule.

An anemic domain model separates data from behavior, often placing all rules in services. That can weaken encapsulation when invariants are scattered, though a simple data-transfer object or CRUD-focused service does not automatically need a rich domain model. Put rules where they can be consistently enforced, and use transactions/concurrency controls for invariants that span multiple requests or records.

Abstraction

Abstraction exposes the essential contract while hiding details that callers should not need. For example, a payment workflow can depend on an authorization operation rather than HTTP requests and provider-specific payloads. A good abstraction names a stable capability, specifies failures and expectations, and leaves room for implementations to vary.

Abstraction becomes over-engineering when it adds indirection without a real boundary, creates leaky provider-specific contracts, or anticipates variation that has no evidence. A concrete class can be the clearest choice.

Inheritance

Inheritance lets a derived class reuse and specialize a base class. It is appropriate when the derived type genuinely satisfies the base contract, not simply when two classes share code. Protected members and virtual

hooks couple subclasses to base implementation details. A base-class change can unexpectedly break subclasses—the **fragile base class** problem.

Prefer composition when the variation is an interchangeable behavior, when independent combinations are needed, or when inheritance would require overriding unrelated hooks. Composition is not a ban on inheritance; framework extension points and stable, substitutable hierarchies can justify it.

Polymorphism

Polymorphism allows one contract to represent different implementations. **Runtime polymorphism** selects an overridden virtual or interface implementation based on the actual object. **Compile-time polymorphism** includes overload resolution: the compiler chooses an overload based on the declared argument types and applicable conversions.

In .NET, virtual and interface calls are generally dispatched through runtime type information and dispatch mechanisms. The JIT can devirtualize or inline calls when it can prove the target, so do not assume every call incurs the same indirect dispatch cost. Correct design and measurement matter more than guessing the exact JIT output.

Example

```
using System;

var account = new Account(100m);
account.Withdraw(30m);
Console.WriteLine(account.Balance); // 70

IReceiptSender sender = new EmailReceiptSender();
sender.Send("Payment received");   // runtime polymorphism

sealed class Account
{
    public decimal Balance { get; private set; }
    public Account(decimal openingBalance)
    {
        if (openingBalance < 0) throw new
ArgumentOutOfRangeException(nameof(openingBalance));
        Balance = openingBalance;
    }

    public void Withdraw(decimal amount)
    {
        if (amount <= 0 || amount > Balance)
            throw new ArgumentOutOfRangeException(nameof(amount));
        Balance -= amount;
    }
}

interface IReceiptSender { void Send(string message); }
sealed class EmailReceiptSender : IReceiptSender
{
```

```
    public void Send(string message) => Console.WriteLine($"Email: {message}");
}
```

This in-memory check alone does not prevent concurrent withdrawals from racing; persisted systems also need a suitable concurrency and transaction strategy.

Likely interview questions

1. Explain each OOP pillar using a concrete production example.
2. How can public properties undermine encapsulation?
3. When is a simple data model appropriate, and when does it scatter important invariants?
4. What makes an abstraction useful rather than speculative?
5. Contrast overload resolution with virtual dispatch.
6. What is the fragile base class problem?
7. When would composition be a better fit than inheritance?
8. How would you protect an invariant against concurrent requests?

Pitfalls and recap

- **Pitfalls:** Treating private fields as sufficient without enforcing rules; describing abstraction as “use an interface everywhere”; assuming inheritance is always wrong; confusing overloads with overrides.
- **Recap:** Encapsulate invariants, abstract stable capabilities, inherit only for a valid subtype relationship, and use polymorphism to vary behavior through a contract.

Day 3 — Interface vs abstract class, composition vs inheritance, virtual/override/new

Month 1 — Foundations, Week 1 — C# Fundamentals.

Goal: Choose the right extension mechanism and explain its effect on dispatch and coupling.

Interface vs abstract class

An **interface** describes a contract that unrelated types can implement, and a class can implement multiple interfaces. Interfaces can include default implementations and static members, but they do not supply ordinary per-instance fields or instance constructors. Default interface implementations can help evolve a published contract, but introduce versioning and dispatch considerations; they are not a substitute for shared instance state.

An **abstract class** can hold instance state, define constructors and protected members, and provide shared implementation or required abstract members. A class can inherit only one base class. Choose an interface for a capability or dependency boundary; choose an abstract base when implementations share a meaningful stateful foundation and a genuine subtype relationship. An interface and an abstract base can be used together.

Composition vs inheritance

Composition makes behavior a collaborator: for example, an order processor receives a pricing policy rather than inheriting from `DiscountedOrderProcessor`. This lets pricing, tax, and notification vary independently

and keeps the processor's lifecycle explicit. It also introduces extra objects and wiring; keep the design proportional to the real variation.

To refactor a deep hierarchy, first characterize existing behavior with tests. Identify each varying responsibility, extract a focused collaborator, delegate one behavior at a time, and verify externally observable behavior after each step. Preserve invariants and account for subclass-dependent callers before removing inheritance.

`virtual`, `override`, and `new`

`virtual` declares a member eligible for runtime overriding. `override` replaces an inherited virtual member in the same dispatch chain. `new` **hides** an inherited member; selection depends on the compile-time reference type, which can make behavior surprising. `sealed override` stops further overrides of that member.

Calling a virtual member from a base constructor is risky: dispatch can reach a derived override before the derived constructor has completed, so derived invariants may not hold. Prefer explicit initialization after construction or nonvirtual constructor logic.

Example

```
using System;

Base overridden = new Overriding();
Base hidden = new Hiding();
Console.WriteLine(overridden.Describe());      // override
Console.WriteLine(hidden.Describe());          // base
Console.WriteLine(((Hiding)hidden).Describe()); // hidden

var checkout = new Checkout(new TenPercentDiscount());
Console.WriteLine(checkout.Total(100m));      // 90.0

class Base { public virtual string Describe() => "base"; }
class Overriding : Base { public override string Describe() => "override"; }
class Hiding : Base { public new string Describe() => "hidden"; }

interface IDiscount { decimal Apply(decimal price); }
sealed class TenPercentDiscount : IDiscount
{
    public decimal Apply(decimal price) => price * 0.9m;
}
sealed class Checkout
{
    private readonly IDiscount discount;
    public Checkout(IDiscount discount) => this.discount = discount;
    public decimal Total(decimal price) => discount.Apply(price);
}
```

Likely interview questions

1. Give a concrete rule for choosing an interface or an abstract class.
2. Can an interface have instance fields, an instance constructor, or a default method implementation?

3. When is a default interface method useful, and what compatibility concerns remain?
4. What breaks when unrelated variations are encoded in a deep hierarchy?
5. Explain the output when a hidden member is called through a base reference.
6. Why is a virtual call in a base constructor dangerous?
7. How would you safely refactor an inheritance hierarchy toward composition?
8. Describe an actual decision from your own work without inventing details.

Pitfalls and recap

- **Pitfalls:** Assuming `new` overrides; inheriting merely to reuse code; claiming interfaces cannot contain any implementation; invoking virtual behavior before derived initialization.
- **Recap:** Interface = contract; abstract base = shared state/behavior plus a subtype relationship. `override` participates in dynamic dispatch; `new` hides.

Day 4 — Collections: IEnumerable, ICollection, IList, List, Dictionary, HashSet

Month 1 — Foundations, Week 1 — C# Fundamentals.

Goal: Choose a contract based on the operations needed and reason about performance and evaluation.

Collection interfaces

`IEnumerable<T>` supports iteration. It does not promise a count, indexing, repeatability, or that iteration is cheap. A sequence may read a database or file, execute side effects, or produce different results on subsequent enumeration.

`ICollection<T>` adds count and collection operations such as `Add` and `Remove`; a particular implementation may be read-only and reject mutation. `IList<T>` adds indexed access and positional insertion/removal. Use the narrowest contract that accurately expresses what the method needs. `IReadOnlyCollection<T>` and `IReadOnlyList<T>` are useful when callers need count or indexing without mutation through that interface; they do not make the underlying data immutable.

List, dictionary, and set

`List<T>` is a resizable array. Indexed access is O(1); append is **amortized O(1)**, but a resize copies elements and makes an individual append O(n). Insertion or deletion near the beginning is O(n) because later elements shift. A capacity estimate can reduce resizing but is not a substitute for choosing the right data structure.

`Dictionary<TKey,TValue>` uses key hashes and equality to locate entries. Lookup, insertion, and removal are typically expected O(1), but collisions, resizing, expensive hash/equality implementations, and adversarial inputs can affect cost. Equal keys must have equal hash codes. Do not mutate a key's fields used for hashing or equality while it is in a dictionary: it may no longer be found.

`HashSet<T>` applies similar hashing to unique values and is useful for membership checks and set operations. A list can be better for a tiny collection, positional access, or preserving duplicates. Do not rely on dictionary or set iteration order as an application contract; sort explicitly when order matters.

`IEnumerable<T>` and `IQueryable<T>`

`IQueryable<T>` extends `IEnumerable<T>` but represents a query using expression trees for a provider to translate. With EF Core, filtering before materialization can execute in the database; after `AsEnumerable()` or materialization, subsequent LINQ operators run in the application. Provider translation, database indexes, round trips, and data volume determine the real cost. Execution usually occurs on enumeration or a terminal operator such as `ToList`, `Count`, or an async EF equivalent.

Example

```
using System;
using System.Collections.Generic;
using System.Linq;

var names = new List<string> { "Ada", "Grace", "Ada" };
var unique = new HashSet<string>(names, StringComparer.OrdinalIgnoreCase);
Console.WriteLine(unique.Contains("ADA")); // True

var counts = new Dictionary<string, int>(StringComparer.OrdinalIgnoreCase);
foreach (string name in names)
    counts[name] = counts.GetValueOrDefault(name) + 1;
Console.WriteLine(counts["Ada"]); // 2

IEnumerable<int> query = Enumerable.Range(1, 5).Where(n => n % 2 == 0);
List<int> snapshot = query.ToList(); // materialize once when a snapshot is needed
Console.WriteLine(string.Join(", ", snapshot)); // 2, 4
```

Likely interview questions

1. Which parameter type would you choose for a method that only iterates its input?
2. Why might enumerating the same sequence twice be expensive or produce different results?
3. Compare `List<T>.Add` and `Insert(0, item)` complexity.
4. What assumptions underlie expected O(1) dictionary lookup?
5. Why must key equality and hash codes remain consistent?
6. When would you prefer a `HashSet<T>` to a list?
7. Should a program depend on dictionary iteration order?
8. How can switching from `IQueryable<T>` to `IEnumerable<T>` affect an EF Core query?

Pitfalls and recap

- **Pitfalls:** Assuming every `IEnumerable<T>` is an in-memory collection; calling every list append worst-case O(1); mutating hash keys; materializing a large query before filtering.
- **Recap:** Pick the operations your API needs. Lists offer indexing, dictionaries map keys to values, and sets support uniqueness and membership. Distinguish database-side queries from in-memory iteration.

Day 5 — Generics, generic constraints, covariance/contravariance

Month 1 — Foundations, Week 1 — C# Fundamentals.

Goal: Use generics to express reusable, type-safe code and explain variance without compromising type safety.

Generics and runtime behavior

Generics let code work across types while preserving type information and enforcing type safety. `List<int>` stores integers without converting each element to `object`, avoiding the boxing required by a non-generic `ArrayList`. This does not mean boxing can never occur elsewhere in generic code.

The CLR retains generic type information at runtime. The runtime often shares generated machine code among compatible reference-type instantiations and generates specialized code for value-type instantiations, which may have different sizes and layouts. This is a useful mental model, not an absolute promise about JIT/AOT implementation or every possible optimization.

Constraints

Constraints specify the capabilities generic code may rely on and reject incompatible type arguments at compile time. Examples include `where T : class`, `struct`, a base class or interface, and `new()` for an accessible public parameterless constructor. `new()` comes last among ordinary constraints, and cannot be combined with `struct` or `unmanaged`, which already imply the relevant construction capability. Constraints combine: `where T : class, IComparable<T>, new()` requires all three conditions.

Use `new()` only when generic code truly owns construction; a factory or dependency injection is often more flexible if creation requires configuration or dependencies. `notnull` and nullable-reference annotations have compiler-specific warning behavior; they are not a substitute for runtime validation of untrusted data.

Covariance and contravariance

Variance applies to **interfaces and delegates**, and variance conversions apply to reference types, not value types.

- **Covariance (`out`)** allows a producer of a more derived type to be used where a producer of a base type is expected: `IEnumerable<Dog>` → `IEnumerable<Animal>`.
- **Contravariance (`in`)** allows a consumer of a base type to be used where a consumer of a more derived type is expected: `Action<Animal>` → `Action<Dog>`.
- **Invariance** is needed when a type both consumes and produces `T`. `IList<T>` can return and accept elements, so `IList<Dog>` cannot safely become `IList<Animal>`: the latter would allow adding a `Cat`.

An `IEnumerable<Animal>` reference to a `List<Dog>` does not expose `Add`. A caller could attempt to cast to a mutable interface, but `IList<Animal>` is not a valid covariant conversion from `List<Dog>`; a failed cast or explicit other unsafe mechanisms are separate questions.

Example

```
using System;
using System.Collections.Generic;

IEnumerable<Dog> dogs = new List<Dog> { new Dog() };
IEnumerable<Animal> animals = dogs;      // covariance
Action<Animal> inspect = a => Console.WriteLine(a.GetType().Name);
Action<Dog> inspectDog = inspect;        // contravariance
inspectDog(new Dog());                   // Dog
```

```
var created = Create<Dog>();
Console.WriteLine(created.GetType().Name);

static T Create<T>() where T : Animal, new() => new T();

class Animal { }
class Dog : Animal { public Dog() { } }
```

Likely interview questions

1. Why does `List<int>` avoid boxing in ordinary element storage?
2. How does generic type information differ from a simple compile-time-only template?
3. Why might value-type and reference-type generic instantiations have different generated code?
4. What does `new()` guarantee, and when is a factory better?
5. Explain `out` and `in` with a producer and a consumer.
6. Why is `IList<T>` invariant?
7. Do variance conversions work for `IEnumerable<int>` to `IEnumerable<object>`?
8. How do multiple constraints combine?

Pitfalls and recap

- **Pitfalls:** Saying generics eliminate every possible box; reversing `in` and `out`; assuming variance applies to value-type conversions; claiming constraints replace runtime validation.
- **Recap:** Generics preserve type safety, constraints express required capabilities, `out` produces, `in` consumes, and mutable producer/consumer interfaces are generally invariant.

Day 6 — Delegates: Func, Action, Predicate, events

Month 1 — Foundations, Week 1 — C# Fundamentals.

Goal: Explain callable values, event ownership, and subscription lifetime.

Delegates

A delegate is a type-safe object representing a callable method and, for an instance method, its target. A lambda can be converted to a compatible delegate; capturing a local may require a compiler-generated closure that keeps captured state alive. Two delegates with the same signature but different delegate types are not automatically interchangeable as variables.

`Action` represents a void-returning operation, `Func` represents an operation with a return value, and `Predicate<T>` specifically represents a `T` → `bool` test. `Predicate<T>` and `Func<T, bool>` have the same shape but are different delegate types. Choose the type expected by the API; use a named custom delegate when it makes a domain contract clearer or needs a signature such as `ref` parameters.

LINQ-to-Objects `Where` takes a `Func<T, bool>` and evaluates it as the sequence is enumerated. `Queryable.Where`, by contrast, takes an expression tree so a provider can inspect and translate it. Deferred evaluation means the predicate can run more than once if the sequence is enumerated more than once.

Multicast delegates and events

Combining delegates creates an invocation list. On ordinary invocation, handlers are called in order. If a handler throws and the exception is not handled, later handlers are not called. For a non-void multicast delegate, the invocation expression returns the **last** handler's return value; prior return values are not automatically aggregated. Enumerate `GetInvocationList()` explicitly if every result or independent exception handling is required.

An `event` restricts access to the underlying delegate: outside the declaring type, consumers normally subscribe and unsubscribe with `+=` and `-=`, but cannot invoke it or replace its entire invocation list. This protects the publisher's ownership of notifications.

The publisher holds references to its subscribers through the delegate invocation list. If the publisher outlives a subscriber, forgetting to unsubscribe can keep the subscriber alive. Use a clear ownership/lifetime strategy, unsubscribe when appropriate, and be careful with anonymous lambdas: retain the delegate instance if you need to remove that particular subscription.

Example

```
using System;
using System.Linq;

var source = new TemperatureSensor();
EventHandler<TemperatureEventArgs> handler =
    (_, e) => Console.WriteLine($"Temperature: {e.Value}");
source.Changed += handler;
source.Set(25);
source.Changed -= handler;

int[] readings = { 18, 25, 30 };
Func<int, bool> isWarm = x => x >= 25;
Console.WriteLine(string.Join(", ", readings.Where(isWarm))); // 25, 30

sealed class TemperatureEventArgs : EventArgs
{
    public TemperatureEventArgs(int value) => Value = value;
    public int Value { get; }
}
sealed class TemperatureSensor
{
    public event EventHandler<TemperatureEventArgs>? Changed;
    public void Set(int value) => Changed?.Invoke(this, new
TemperatureEventArgs(value));
}
```

Likely interview questions

1. What does a delegate contain conceptually, and what can a capturing lambda keep alive?
2. Compare `Action`, `Func<T, bool>`, and `Predicate<T>`.
3. What happens when a multicast handler throws?
4. Which result is returned by a multicast delegate with a return type?

5. How does an event differ from a public delegate field?
6. How can event subscriptions cause a memory leak?
7. How do `Enumerable.Where` and `Queryable.Where` differ?
8. When would a custom delegate be clearer than `Func` or `Action`?

Pitfalls and recap

- **Pitfalls:** Assuming all handlers run after an exception; assuming multicast results are aggregated; forgetting subscription lifetime; confusing a delegate with an expression tree.
- **Recap:** Delegates represent callable behavior; events control who may invoke or replace notification handlers. Manage subscriptions deliberately.

Day 7 — Review + ~15 interview questions, no notes

Month 1 — Foundations, Week 1 — C# Fundamentals.

Goal: Answer without notes, connect topics, and turn experience into an accurate project story.

Review plan

1. Explain each topic aloud in 60–90 seconds without looking at the notes.
2. Answer the questions below; write down where your explanation was imprecise or depended on memorized phrasing.
3. For each weak answer, revisit the relevant section, give a small example, and try again without notes.
4. Practice the project story below. Use only facts you can substantiate.

Fifteen interview questions

1. A method receives a class instance by value. Why can it change an object's property but not replace the caller's variable?
2. Where can a value type live besides a stack frame?
3. What is boxed when an `int` is added to an `ArrayList`, and what happens during unboxing?
4. What criteria would make you choose an immutable struct instead of a class?
5. Give a practical example of encapsulation protecting an invariant.
6. When does adding an abstraction become unnecessary indirection?
7. Contrast overload selection, `override`, and `new`.
8. When do you choose an abstract class over an interface?
9. How would you replace a fragile inheritance hierarchy with composition without breaking callers?
10. Why can enumerating an `IEnumerable<T>` twice be costly or surprising?
11. Explain expected dictionary lookup complexity and the consequences of mutating a key.
12. What happens if an EF Core query switches to in-memory enumeration before filtering?
13. Explain covariance and contravariance with concrete C# types.
14. Why is `IList<Dog>` not safely assignable to `IList<Animal>`?
15. How can an event subscription keep an object alive, and how would you prevent it?

Worked review example

Try the questions first without consulting notes. Then use this example to connect copying, encapsulation, composition, collections, generics, delegates, and events:

```
using System;
using System.Collections.Generic;

var original = new Price(100m);
var copy = original;
copy.Amount = 80m;
Console.WriteLine(original.Amount); // 100: struct assignment copied the value

var checkout = new Checkout(new TenPercentDiscount());
EventHandler<PriceEventArgs> handler =
    (_, e) => Console.WriteLine($"Total: {e.Amount}");
checkout.Completed += handler;
checkout.Complete(original);
checkout.Completed -= handler;

var totals = new Dictionary<string, decimal> { ["order-1"] = 90m };
Console.WriteLine(totals["order-1"]); // 90

struct Price { public Price(decimal amount) => Amount = amount; public decimal
Amount; }

interface IDiscount { decimal Apply(decimal amount); }
sealed class TenPercentDiscount : IDiscount
{
    public decimal Apply(decimal amount) => amount * 0.9m;
}

sealed class PriceEventArgs : EventArgs
{
    public PriceEventArgs(decimal amount) => Amount = amount;
    public decimal Amount { get; }
}

sealed class Checkout
{
    private readonly IDiscount discount;
    public Checkout(IDiscount discount) => this.discount = discount;
    public event EventHandler<PriceEventArgs>? Completed;
    public void Complete(Price price)
    {
        decimal total = discount.Apply(price.Amount);
        Completed?.Invoke(this, new PriceEventArgs(total));
    }
}
```

`Checkout` composes an interface-based discount rather than inheriting a specific discount; an event delegates notification to subscribers. The generic dictionary stores its `decimal` values without ordinary element boxing. Unsubscribe when the subscription lifetime ends, especially when the publisher can outlive the subscriber. This minimal example illustrates the mechanisms, not production payment validation or persistence.

Aloha Cloud architecture: five-minute story

Prepare a five-minute explanation of your actual Aloha Cloud architecture. Do not invent components or impact. If a detail is confidential, describe it at a safe level of abstraction.

1. **Context and users:** What problem did the system solve, and for whom?
2. **Architecture and flow:** What were the major components, and what happened from request to response?
3. **Your ownership:** Which components or decisions were yours?
4. **Tradeoff:** Identify one real decision, alternatives considered, and why the chosen approach fit the constraints.
5. **Production reality:** Describe one verified challenge, how the team diagnosed it, and its measurable outcome if known.
6. **Reflection:** What would you improve now, and why?

Practice template: "The system served [users] by [purpose]. My responsibility was [actual scope]. A request flowed through [actual components]. We chose [decision] over [alternative] because [constraints]. When [real issue] occurred, I [action] and observed [verified result]. In hindsight, I would [improvement]."

Common pitfalls and recap

- **Pitfalls:** Reciting definitions without an example; implying one data structure or design pattern is always best; reporting imagined project outcomes; omitting tradeoffs and failure modes.
- **Recap:** Explain the mechanism, show a concrete example, state when it is useful, and mention the important limitation. For personal stories, separate your own work from the team's and be honest about unknowns.